React useMemo Tutorial

Use `useMemo` to prevent an expensive calculation from running on unrelated re-renders.

Learning Goals

1. Define what `useMemo` does.
2. Explain dependency arrays in memoization.
3. Build a working "expensive calculation" demo.
4. Apply the same idea to real class code (Context provider values).

---

1) What is useMemo?

`useMemo` is a React Hook that stores a computed value and recalculates it only when dependencies change.

Core pattern:

```
const cachedValue = useMemo(() => computeSomething(), [dep1, dep2]);
```

Meaning:

1. Initial render: run the function and store result.
2. Later renders: reuse cached result if dependencies did not change.
3. If a dependency changed: recompute and update cache.

---

2) Why this matters

A component can re-render for many reasons.

If a heavy calculation sits directly in render logic, it runs every time, even when unrelated state changes.

`useMemo` helps when:

1. The calculation is expensive.
2. You want to skip repeated work on unrelated renders.

---

3) Class Demo

In this demo, we keep two counters:

1. `count` changes often.
2. `otherCount` is unrelated.

We attach an expensive calculation to `count` only.

Step 1: Start component state

```
const [count, setCount] = useState(0);
const [otherCount, setOtherCount] = useState(0);
```

Step 2: Create expensive function

```
function expensiveCalculation(num) {
  console.log("Running expensive calculation...");
  for (let i = 0; i < 1000000000; i += 1) {
    // Simulate heavy CPU work
  }
  return num * 2;
}
```

Step 3: Memorize the calculation

```
const doubledCount = useMemo(() => expensiveCalculation(count), [count]);
```

Step 4: Render controls

```
<button onClick={() => setCount((c) => c + 1)}>Count + 1</button>
<button onClick={() => setOtherCount((c) => c + 1)}>Other + 1</button>

<p>Count: {count}</p>
<p>Other Count: {otherCount}</p>
<p>Expensive Result: {doubledCount}</p>
```

Step 5: Observe behavior

1. Click `Other + 1`.
2. Component re-renders, but expensive calculation does not run again.
3. Click `Count + 1`.
4. Expensive calculation runs because the dependency changed.

---

4) Complete Runnable Example

```
const { useMemo, useState } = React;

function App() {
  const [count, setCount] = useState(0);
  const [otherCount, setOtherCount] = useState(0);

  function expensiveCalculation(num) {
    console.log("Running expensive calculation...");
    for (let i = 0; i < 1000000000; i += 1) {
      // Simulate heavy CPU work
    }
    return num * 2;
  }

  const doubledCount = useMemo(() => expensiveCalculation(count), [count]);

  return (
    <main>
      <h1>useMemo Demo</h1>
      <button onClick={() => setCount((c) => c + 1)}>Count + 1</button>
      <button onClick={() => setOtherCount((c) => c + 1)}>Other + 1</button>

      <p>Count: {count}</p>
      <p>Other Count: {otherCount}</p>
      <p>Expensive Result: {doubledCount}</p>
    </main>
  );
}
```

---

5) Connect to Your Context Lesson

In your Context demo, this is a practical use case:

```
const value = useMemo(
  () => ({
    studentName,
    course,
    changeStudent: setStudentName,
    changeCourse: setCourse,
  }),
  [studentName, course]
);
```

Why:

1. Provider `value` is an object.
2. Without `useMemo`, object reference changes each render.
3. Stable references can reduce unnecessary downstream updates.

---

6) Common Mistakes

1. Missing dependencies (causes stale values).
2. Adding `useMemo` to tiny/cheap calculations.
3. Assuming `useMemo` stops all component re-renders.

Remember: `useMemo` memoizes values, not whole components.

---

7) Class Challenge

Task

Build a student score panel where filtering and sorting are memoized.

Requirements:

1. Search by student name.
2. Toggle `Passing only` (score >= 60).
3. Sort descending by score.
4. Add an unrelated state button (`Theme Toggle` or `Clicks`) to prove memoization.

---

8) Step-by-Step Solution

Step 1: State setup

```
const [search, setSearch] = useState("");
const [passingOnly, setPassingOnly] = useState(false);
const [uiClicks, setUiClicks] = useState(0); // unrelated state
```

Step 2: Data source

```
const students = [
  { id: 1, name: "Linus", score: 72 },
  { id: 2, name: "Ada", score: 95 },
  { id: 3, name: "Grace", score: 58 },
  { id: 4, name: "Alan", score: 83 },
];
```

Step 3: Memoized filtered/sorted list

```
const visibleStudents = useMemo(() => {
  console.log("Recomputing student ranking...");
  return students
    .filter((s) => s.name.toLowerCase().includes(search.toLowerCase()))
    .filter((s) => (passingOnly ? s.score >= 60 : true))
    .sort((a, b) => b.score - a.score);
}, [students, search, passingOnly]);
```

Step 4: Controls

```
<input value={search} onChange={(e) => setSearch(e.target.value)} />
<label>
  <input
    type="checkbox"
    checked={passingOnly}
    onChange={(e) => setPassingOnly(e.target.checked)}
  />
  Passing only
</label>

<button onClick={() => setUiClicks((n) => n + 1)}>
  Unrelated UI Clicks: {uiClicks}
</button>
```

Step 5: Render results

```
<ul>
  {visibleStudents.map((s) => (
    <li key={s.id}>{s.name} - {s.score}</li>
  ))}
</ul>
```

Step 6: Verification checklist

1. Search text changes recalculate list.
2. Passing toggle recalculates list.
3. Unrelated button does not recalculate list.

---

9) Notes

1. Watch console logs while clicking buttons.